

Sol Current-State Audit

Production and Maintenance Plan

Document-based assessment through P2.6
September 9, 2026 | Implementation baseline: 6ec4ba9

1 Executive Assessment

Sol is an experimental language and C17 compiler with a substantial executable reference model, not a production-ready compiler or stable toolchain. Explicit domain types, ownership, effects, capability authority, contracts, and semantic identity form a coherent foundation for inspectable software change. Whether they make human or AI maintenance safer, faster, or cheaper remains unmeasured.

IMPLEMENTED E1-E6, P1, and P2.1-P2.6 are complete at the documented baseline. Reference application execution and runtime callable/refinement checks are operational. Separate internal production owners now reach concrete specialization, representation, target layout, and semantic-operation plans.

OPEN P2 overall remains open. P2.7 is next overall and next production. There is no backend, production runtime ABI, sol build, full public IR, or SMT proof discharge. Wasm is the first production target, not delivered output.

The recommended strategy preserves the frozen executable core while testing a separate product hypothesis: can bounded semantic context and protected review improve realistic maintenance? Production delivery alone cannot answer that. The proposed first-domain direction is a capability-restricted hosted business-logic or validation component; M1 must still choose its user, workload, and host boundary. This audit selects no workload and creates no language prerequisite.

1.1 Scope and Source Authority

This refresh replaces the stale August 25 assessment rather than appending a checkpoint history. It reconciles documentation, not fresh code inspection, compiler execution, security testing, or release certification. Baseline evidence is reported evidence, not a September 9 test result.

Source	Authority and Use
TODO.md	Sole live status, exact execution cursor, dependencies, stable task IDs, and acceptance criteria.
README.md	Current introduction, runnable commands, supported scope, and evidence labels.
Compiler status	Detailed 6ec4ba9 implementation and compatibility snapshot; includes historical evidence.
Project analysis	Document-based product assessment, historical findings, and experiment rationale.

Use the [design manual](#) for target-language design and its distinction from implemented scope. This audit is a dated orientation, not another work ledger. Links are relative to the repository-root PDF; retain that location when using them.

2 Executable Baseline and Evidence

2.1 Reference Execution

The normal compilation session owns phase order, bounded source/package loading, diagnostics, and frontend teardown. Successful compilation transfers self-contained owning typed IR through an opaque validated handle. The interpreter executes that IR after frontend state is freed; it does not execute the separate P1/P2 owners.

```
source/package -> syntax validation -> resolution/type/effect analysis
-> owning typed IR -> ownership/final validation -> opaque handle
-> reference execution: sol test / sol run
```

The implemented CLI provides check, test, run, effects, inspect, and fmt. Directory inputs form one deterministic dependency-free package with multi-file modules and explicit public imports, not manifests or downloaded dependencies. The bounded core includes records, enums, tuples, nominal types, Option/Result, bounded generics and traits, recursive matching, checked mutation, partial moves, lexical borrows/regions, loops, and exact handlers.

Application execution resolves one explicit @entry: a public nongeneric free function with owned root-capability parameters, explicit closed effects, and () or Int64 result. Unit maps to status 0; integer success must be in 0-255. The trusted host grants exact root/member operations for bounded console output, argument access, and configuration snapshots. There is no ambient filesystem, network, clock, randomness, console input, or live environment access.

2.2 Runtime Contracts Are Operational

SOL_INTERPRETER_CONTRACTS_CHECK evaluates requires, captures entry-state old, and evaluates applicable ensures after a valid return, including Result outcomes and approved bodyless hosted-member contracts. False predicates have structured contract diagnostics; predicate runtime failures retain their failure identity. Checks share execution limits and preserve ordinary owned-binding cleanup. sol run and sol test request CHECK; explicit library callers may select IGNORE. Direct refined construction always evaluates its type-owned predicate.

Loop invariants and decreases are typed, pure obligation templates but remain runtime-erased and unproved. Proof-backed unreachable retains an unresolved obligation; reaching it produces a defensive runtime error. No solver discharge is supplied by accepting this syntax or by materializing a predicate CFG.

Evidence Label	Required Interpretation
Declared	Source states intent. A well-typed predicate is not a proof.
Runtime-checked	A supported check ran on a particular execution under its policy. It does not establish all-input behavior.
Proved-under-assumptions	A proof covers an explicit property, model, and trusted assumptions. The bootstrap does not provide SMT discharge.
Unknown	Unresolved, unsupported, omitted, or outside the modeled boundary. Unknown is neither success nor an empty dependency set.

pure does not mean total, terminating, or automatically proved. E6 conformance, focused MIR-vs-owning-IR comparisons, malformed-input tests, and exact censuses support bounded compiler claims, not usability or general correctness. See the [baseline evidence snapshot](#) for historical test reports; no compiler suite was rerun for this document.

3 Production Internals Through P2.6

The completed checkpoints are meaningful compiler progress, but remain unstable internal APIs separate from the compilation session and CLI. Their closure is the frozen E6 profile, not all target-language breadth or every reference-interpreter program. Representation and Wasm32 layout are not a runtime ABI or a backend.

Complete	Delivered Internal Boundary
P1	Callable-scoped target-neutral CFG MIR for every bodyful E6 callable; explicit SSA, ownership, storage, failure/cleanup, and source relations; independent validation, canonical rendering, and bounded differential evaluation.
P2.1-P2.2	A deterministic symbolic program and canonical monomorphic instance plan, approved imports and specialization demands, finite closure, substitutions, recursion handling, and bounded construction.
P2.3	Owned concrete callable images, closed dispatch/effects, handlers, normal-edge writeback, and independent SSA/ownership/cleanup revalidation. Symbolic topology remains for authentication, not executable dispatch.
P2.4	Canonical target-neutral recipes for concrete types, aggregate shapes, callable producers, capability sources, Copy/drop classifications, and independently validated arena/resource censuses.
P2.5	Explicit target-parameterized layouts, initial Wasm32 descriptor, checked size/alignment/padding, sum tags/payloads, and projected-place access maps.
P2.6	Representation-aware operations plus source-independent predicate CFGs and import-contract envelopes. Executable records use concrete IDs and operations; source IDs are segregated authenticated provenance.

3.1 Restrictions Remain Part of the Contract

The program owner requires exact direct static definition or bound-operation producers at callback sites. Ordinary first-class body callbacks and dynamic, conditional, or aliased callable producers remain outside that production closure, even where reference execution is broader. Contextless generic, receiver, and effect-polymorphic fixture roots reject without a root-context API. Reachable open enums reject at representation. These restrictions are not silently relaxed by P2.6.

P1 callable contracts still reject contracted generic/effect-parameter or exclusive forms and fallible/projected snapshots; supported snapshots are direct scalar locals. Capability-member bodies have no standalone P1 MIR. Approved bodyless import contracts are nevertheless closed by P2.6 and are not globally unsupported.

P2.6b2 supports rich bounded predicate control flow, exact pure calls, aggregate and refined construction, recursive matches/guards, immutable locals, and nested refinement references. It is no longer scalar-single-block-only and has no UNRESOLVED_BODY state. Contract-expression propagation still rejects under SOL-CONTRACT-002. The small E6 predicate census is a fixture shape, not a limit on all predicate support or a count of the whole test suite.

3.2 Internal Ownership Is Not a Serialization Boundary

Raw owning IR and the mutable P owners are trusted compiler-internal C structures, not full public IR or hostile serialized input. Logical checks cannot establish that arbitrary foreign pointers address readable allocations. Borrowed upstream owners must remain alive and immutable through downstream use and destruction; teardown requires original builder-owned allocation bases. Independent validation does not turn these APIs into arbitrary-address recovery interfaces.

`sol inspect` supplies selected versioned external projections, not these owners, a decoder, complete dependency coverage, or a guaranteed semantic cache key. The [detailed snapshot](#) owns the full compatibility limits.

4 Production and Experiment Order

This sequence mirrors the [live execution cursor](#) at refresh time. Only TODO owns subsequent status and order; completed E milestones are the starting point, not future endpoints.

E1-E6 + P1 + P2.1-P2.6 complete
-> P2.7 (next overall and next production)
-> M1 workload / experiment charter
-> P2.8 concrete-program freeze
-> P3 runtime ABI -> P4 Wasm backend/adaptor -> P5 build/conformance

After M1 feasibility + P2.8:

M2 packets -> M3 deltas -> M4 edit/approval -> M5 held-out comparison

M4E basic editor slice: optional after M1/M2 projection feasibility

M1 publishes subsequent interleaving. If infeasible, record the blocker and continue P2.8 without inventing prerequisites. After P2.8, M2-M5 may proceed independently alongside P3 and the later production path, subject to their own dependencies, staffing, and exclusive file ownership. They are not mutually independent tasks: M3 needs M2, M4 needs M2/M3, and M5 needs M4. One coordinator maintains the single next-overall cursor.

4.1 Finish the Concrete Boundary, Then the Runtime

Open Track	Bounded Outcome
P2.7	Collision-checked ASCII symbols from semantic identity and canonical instance keys; deterministic callable/export/table identities; ordinary callable linkage resolved internally, with only typed runtime and approved-host requirements.
P2.8	Complete concrete-owner validation, canonical rendering, malformed-input coverage, repeated-lowering equality, and a finite E6 closure including failure and cleanup. No ABI selection or Wasm emission.
P3	Backend-neutral call/result/failure conventions, bounded owned-value allocation/operations, cleanup and panic policy, capability/host and exact-handler ABI, then runtime-lowered-program validation.
P4	Pinned established Wasm toolchain, scalar and represented-value emission, runtime checks and cleanup, trusted host adaptor, and E6 Wasm execution.
P5	Production build API and <code>sol build</code> , deterministic artifact writes, artifact execution, interpreter/Wasm differential conformance, and reproducible development/release acceptance.

4.2 P3.W1 Is an Experiment, Not Backend Completion

After P3.1 has tested call/result/failure conventions, timebox a candidate Wasm ABI integration experiment. Pin and report candidate tool versions, execute a minimal scalar call/result/failure module, and record ABI/tool mismatches. Allocation, cleanup, and host imports enter only after the relevant P3.2-P3.4 conventions are tested. This informs tool selection; it does not complete P4.1, backend/adaptor tasks, Component Model support, E6 Wasm execution, or build tooling.

Stable numbered capabilities and completed named slices remain in the [numbered backlog](#). This audit neither rennumbers them nor duplicates their full exit criteria. Frozen-core production work does not activate manifests, libraries, public IR, proof, or new syntax by implication.

5 Bounded Maintenance Experiment

PROPOSED All M milestones are open proposals, not shipped commands or features. Use existing inspection, reference execution, ordinary edits, protected tests, and human approval. Full public IR (54), SMT (52), and a patch DSL (56) are not prerequisites, nor is a completed production backend.

5.1 M1: Workload and Evaluation Charter

Select one user and capability-restricted hosted business-logic/validation workload, with a concrete host boundary and realistic maintenance tasks. Protect the starting baseline and held-out tests. Publish feasibility, a timebox, metrics, stop/go thresholds, explicit non-goals, projection sufficiency/gaps, and subsequent inter-leaving. E6 is a conformance fixture, not the selected user or a usability benchmark.

Prefer a narrow host or library solution to a new language family. If a real blocker appears, propose a separately bounded slice with authority, interface, tests, and acceptance criteria. No charter or workload selection is delivered by this documentation refresh.

5.2 M2-M4: Context, Deltas, and Approval

M2: declaration-centered packets (58 slice). Produce bounded deterministic source, signatures, effects, contracts, and available references. Include a source snapshot hash, compiler/schema/options and selection metadata, and explicit omissions. Unavailable is not empty. Use semantic IDs only within their supported scope; package-local top-level identities do not identify every local/member or cross-dependency entity. Inspection bytes are not a promised semantic cache key.

M3: conservative checked-snapshot deltas (57 slice). Compare declaration, signature, effect, and contract facts after M2. Validate stable identities and base snapshots; reject stale, ambiguous, and unsupported comparisons. Preserve body edits and unknown behavior in the report. A changed predicate is not established weakening; an unchanged interface does not establish preserved behavior.

M4: ordinary-edit validation/approval (56 slice). Exercise edit, check, test, delta, and human review against the protected baseline. Tests and acceptance policy cannot silently weaken. Changes to authority or contracts require explicit approval. This is not patch syntax, automated architectural repair, or a proof that an accepted edit is correct. Generated code has no bypass around compiler, host, test, or approval boundaries.

M4E: optional editor slice (55 slice). After M1/M2 establish projection feasibility, timebox diagnostics, navigation, and packet presentation. It is not full LSP and is not required for M5. The proposed slices do not complete or earn partial-completion status for their broader numbered capabilities.

5.3 M5: Held-Out Comparison

After M4, compare semantic-tool assistance with source-only maintenance on protected held-out tasks. Both conditions retain the same compiler and test access, equivalent starting source and task definitions, and matched time, model, token, and attempt budgets where relevant. Source-only does not mean removing validation. Record separately any explicitly approved requirement changes.

Measure correct-change rate, introduced regressions, reviewer effort, context size, iterations, diagnostic usefulness, and false confidence. Record unknowns and human review outcomes, not just whether generated source compiles. Small demonstrations establish feasibility; held-out work tests generalization.

Report gains, negative results, and inconclusive outcomes against the original thresholds. Decide continue, narrow, or stop rather than moving the success criteria. Production compiler progress can remain worthwhile without claiming a maintenance advantage that the experiment did not establish. Dependencies and acceptance details remain in the [live M track](#).

6 Risks, Decisions, and Deferrals

6.1 Highest-Priority Assessment Risks

Product thesis and scope interaction. A sophisticated compiler or successful Wasm artifact does not establish safer maintenance. The broad language vision can make unrelated work appear prerequisite, while P and M tracks can compete for staff and files. Preserve the frozen profile and one cursor; require workload evidence for additional scope. Novel syntax also brings model-training, examples, and repair-pattern debt: explicitness alone is not AI usability.

Historical documentation drift. The analyzed baseline mixed target design, runtime support, and stale E/P plans. The refreshed ledger and snapshots address that presentation failure; it is not a permanent allegation that contracts are absent. Keep TODO live, other assessments dated, and evidence labels explicit. The corrected manual reservation example distinguishes a new reservation's stock decrement from a same-request retry's unchanged stock. It remains target pseudocode, not implemented or proved transaction semantics.

Guarantee overreach and trust. Keep declared, runtime-checked, proved-under-assumptions, and unknown separate. Pure is not total. Capability permission does not establish business authorization, tenant isolation, or information-flow safety. Safe hosting must preserve exact grants and data-only transfer; raw mutable internal owners remain outside a hostile-input boundary.

6.2 Workload-Gated Semantic and Ergonomic Decisions

Named arguments currently evaluate in canonical formal-parameter order, not written operand order. A signature reorder can change effects. Preserve that behavior now; reconsider alternatives only through an explicit decision and tests. Direct checked refined construction exists. Recoverable Result-based validation and safe representation projection are distinct proposed APIs, not capabilities already supplied by that construction. Neither becomes a new production prerequisite through this audit.

Measure practical text/data needs, Boolean-test limitations, assertions and failure reporting, editor/debugging friction, and installation from a new user's perspective. The bootstrap is POSIX-only; documented setup requires C17, CMake, and Python for default tests, plus Ninja for the README recipe. Test discovery belongs to a configured build directory. This refresh makes no universal CTest failure claim and certifies no clean installation or test run.

6.3 Compiler Investment and Unverified Risks

Preserve deterministic bounds, source provenance, independent validation, transactional failure, and exact cleanup. Historical concerns included raw host failure text, package races, scaling, and repeated immutable-IR validation; the documented E1 work addresses these with owned text, descriptor-relative loading, SCC analysis, resource budgets, and opaque validated handles. They are not reasserted here as confirmed current defects.

Parallel semantic tables, duplicated validation policy, and deep owner dependencies remain architecture risks to investigate, not newly code-audited bugs. Exact arena and resource censuses are useful regression evidence, not user-facing latency or maintenance measurements. Measure edit/check/test/inspect latency, memory, and defect patterns before refactoring validators or freezing incidental traversal counts. Conformance and sanitizer reports cannot substitute for workload usability or a fresh trust-boundary review.

6.4 Keep Broad Language Work Deferred

Do not pull closures, richer traits, broad numerics/collections, lifetime/resource and allocator systems, dependencies/manifests, unsafe/FFI, public IR/SMT, or full editor/patch systems into P2-P5 by implication. Additional backends or a VM, general resumptive handlers, concurrency, workflows/transactions, real-time claims, reflection/build transforms, and mandatory global sorting remain deferred or separately gated. Preserve implemented features; activate only demonstrated bounded needs under the [live workload decisions](#).

Conclusion: close P2.7, charter M1, then freeze P2.8. Continue the bounded production path while independently measuring maintenance value. Deliver evidence and honest unknowns, not broader guarantees than the current boundary supports.